Прикладне програмування на **Python**

БАЛИ

Лабораторні роботи (4x10)	40
Індивідуальна робота 1 Типи даних та анотування	8
Індивідуальна робота 2 Файли та каталоги	8
Індивідуальна робота 3 Елементи теорії ігор і теорії графів	8
Індивідуальна робота 4 Наукові обчислення	10
Тест-колоквіум 13 тестів по 2 бали	26

БАЛИ

40	8	8	8	10	26
Лабораторні роботи (4x10)	**Індивідуальна робота 1** Типи даних та анотування	**Індивідуальна робота 2** Файли та каталоги	**Індивідуальна робота 3** Елементи теорії ігор і теорії графів	**Індивідуальна робота 4** Наукові обчислення	**Тест-колоквіум** 13 тестів по 2 бали

БАЛИ

40	8	8	8	10	26
Лабораторні роботи (4x10)	**Індивідуальна робота 1** Типи даних та анотування	**Індивідуальна робота 2** Файли та каталоги	**Індивідуальна робота 3** Елементи теорії ігор і теорії графів	**Індивідуальна робота 4** Наукові обчислення	**Тест-колоквіум** 13 тестів по 2 бали

БАЛИ

Лабораторні роботи (4x10)	40
Індивідуальна робота 1 Типи даних та анотування	8
Індивідуальна робота 2 Файли та каталоги	8
Індивідуальна робота 3 Елементи теорії ігор і теорії графів	8
Індивідуальна робота 4 Наукові обчислення	10
Тест-колоквіум 13 тестів по 2 бали	26

БАЛИ

Лабораторні роботи (4x10)	40
Індивідуальна робота 1 Типи даних та анотування	8
Індивідуальна робота 2 Файли та каталоги	8
Індивідуальна робота 3 Елементи теорії ігор і теорії графів	8
Індивідуальна робота 4 Наукові обчислення	10
Тест-колоквіум 13 тестів по 2 бали	26

БАЛИ

Лабораторні роботи (4x10)	40
Індивідуальна робота 1 Типи даних та анотування	8
Індивідуальна робота 2 Файли та каталоги	8
Індивідуальна робота 3 Елементи теорії ігор і теорії графів	8
Індивідуальна робота 4 Наукові обчислення	10
Тест-колоквіум 13 тестів по 2 бали	26

БАЛИ

Лабораторні роботи (4x10)	40
Індивідуальна робота 1 Типи даних та анотування	8
Індивідуальна робота 2 Файли та каталоги	8
Індивідуальна робота 3 Елементи теорії ігор і теорії графів	8
Індивідуальна робота 4 Наукові обчислення	10
Тест-колоквіум 13 тестів по 2 бали	26

БАЛИ

Лабораторні роботи (4x10)	40
Індивідуальна робота 1 Типи даних та анотування	8
Індивідуальна робота 2 Файли та каталоги	8
Індивідуальна робота 3 Елементи теорії ігор і теорії графів	8
Індивідуальна робота 4 Наукові обчислення	10
Тест-колоквіум 13 тестів по 2 бали	26

РЕДАКТОРИ КОДУ ТА ІНТЕГРОВАНІ СЕРЕДОВИЩА РОЗРОБКИ (IDE)

РЕДАКТОРИ КОДУ ТА ІНТЕГРОВАНІ СЕРЕДОВИЩА РОЗРОБКИ (IDE)

- *Редактори коду* — це програми для обробки коду із додатковими функціями. Серед них форматування коду, автодоповнення, підсвічування синтаксису.

- У *IDE (інтегрованого середовища розробки*) функціонал ширший, ніж у редакторів, але воно потребує більших потужностей системи.

- Середовище розробки Python зазвичай складається з редактора коду, налагоджувача та компілятора. Є IDE тільки для Python, але більшість працює з кількома мовами програмування.

Top
Python
IDEs
PyDev
PyCharm
Spyder
Jupyter
Notebook
Rodeo
Thonny
Microsoft
Visual Studio
eric
Wing
Atom

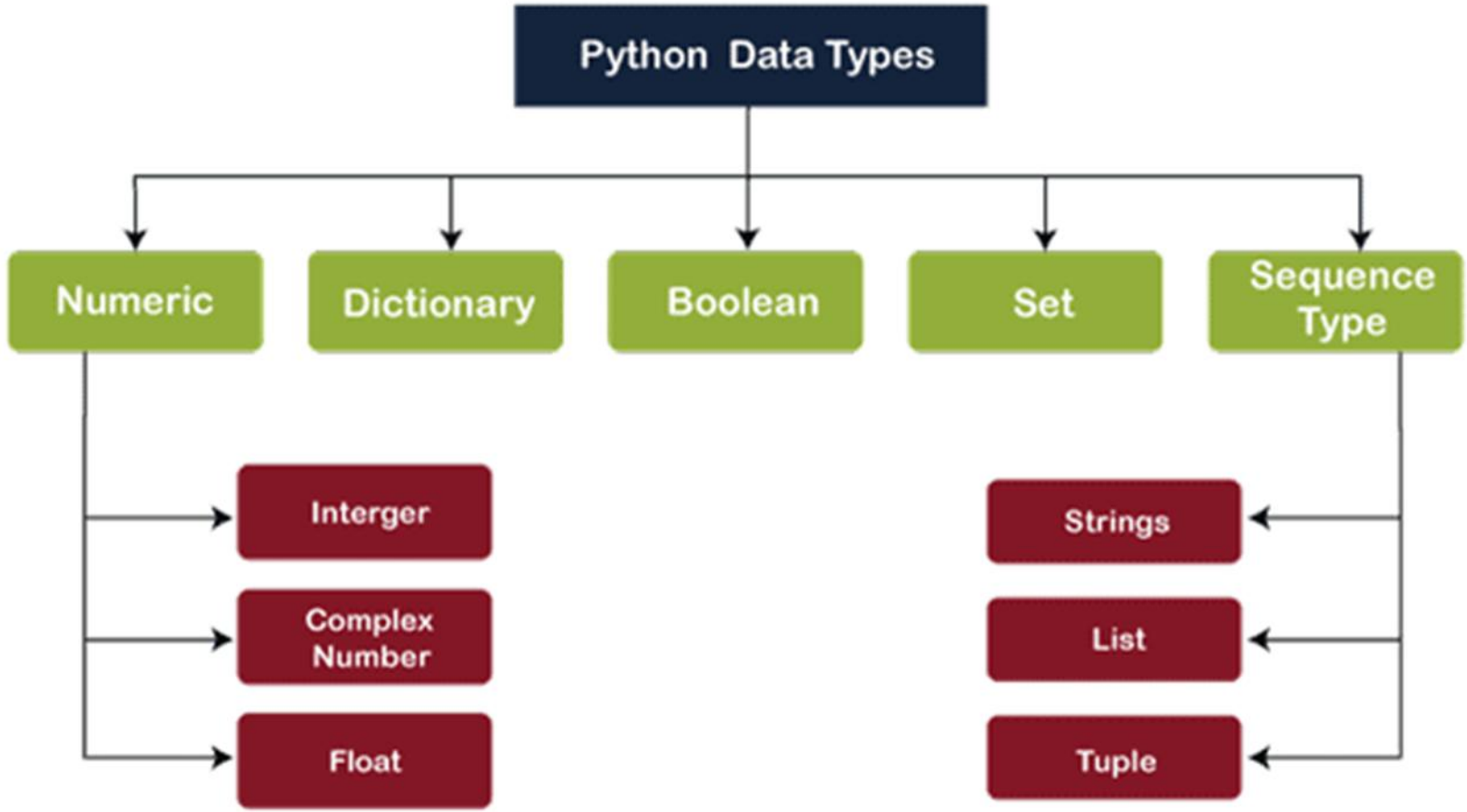
Запуск програм
IDE використовує інтерпретатор Python, встановлений у системі

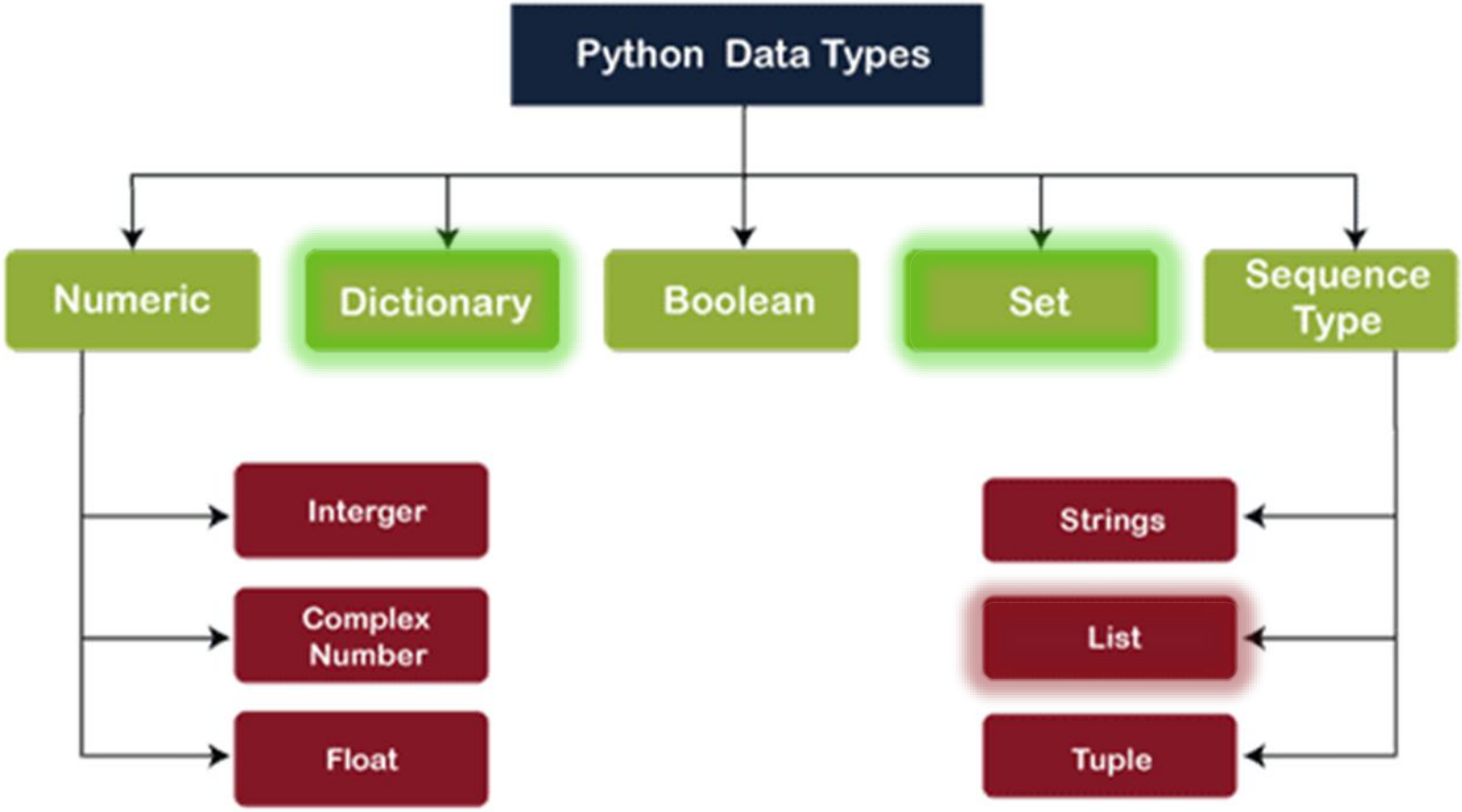
Файлова система
IDE дозволяє читати/записувати файли, створювати проєкти

Віртуальне середовище
Використовується venv або conda для ізоляції залежностей

Системні команди
Деякі IDE інтегруються з терміналом (cmd, bash) для виконання команд Python

Бібліотеки
Автоматична установка пакетів через pip або conda


Python Data Types
Numeric
Dictionary
Boolean
Set
Sequence Type
Interger
Complex Number
Float
Strings
List
Tuple


Python Data Types
Numeric
Dictionary
Boolean
Set
Sequence Type
Interger
Complex Number
Float
Strings
List
Tuple

Основні оператори

Арифметичні

+, -, *, /, //, %, **

Порівняння

==, !=, <, <=, >, >=

Присвоєння

=, +=, *=, ...

Логічні

and, or, not

```
6/2/4  # ?
6/(2/4)  # ?
2**2**3  # ?
(2**2)**3  # ?
```

```
a = 0.01
b = 0.1**2
a == b  # ?
```

```
a = 0
a or 4 < 3  # ?
(a - 2 or a) and 4 <= 4  # ?
not a + 1  # ?
a or 1  # ?
```

```
var = ('Це або дуже-дуже-дуже-дуже-дуже довгий рядок '
       'або кортеж. Що скажете?')

type(var)  # ?
```

```
var1  = "кортеж - це впорядкована та скінченна сукупність елементів! "\
  'а рядок - незмінний упорядкований тип даних, який записується як'\
  'послідовність символів у одинарних чи подвійних лапках;'
type(var1)  # ?
```

```
6/2/4  # 0.75
6/(2/4)  # 12.0
2**2**3  # 256
(2**2)**3  # 64
```

```
a = 0.01
b = 0.1**2
a == b  # False
```

```
a = 0
a or 4 < 3  # False
(a - 2 or a) and 4 <= 4  # True
not a + 1  # False
a or 1  # 1
```

```
var = ('Це або дуже-дуже-дуже-дуже-дуже довгий рядок '
          'або кортеж. Що скажете?')

type(var)  # str
```

```
var1  = "кортеж - це впорядкована та скінченна сукупність елементів! "\
  'а рядок - незмінний упорядкований тип даних, який записується як'\
  'послідовність символів у одинарних чи подвійних лапках;'
type(var1)  # str
```

Ескейп-послідовності

Часто бувають ситуації, коли у змінній рядкового типу `str` потрібно записати, наприклад символ ", або \, або може навіть і деякий символ, який потрібно задати через кодування Unicode (наприклад é).

У такому випадку зручно використати синтаксис ескейп послідовностей.

- \' - одинарна лапка;
- \" - подвійна лапка;
- \\ - бекслеш (\);
- \n - перехід на новий рядок (Enter);
- \r - переміщення каретки на початок рядка;
- \b - переміщення на один символ уліво зі стиранням (Backspace);
- \t - горизонтальна табуляція (TAB);
- \v - вертикальна табуляція;
- \u, \U, \N{} - символ у кодуванні Unicode;
- \x - відтворює символ за шістнацядковим кодом.

```
s = 'Це розповідне речення.'
new_s = s+'\b'
print(new_s)  # print обов'язковий!
```

Змінювані (Mutable) та Незмінювані (Immutable) типи даних

Незмінювані об'єкти ніколи не змінюються після створення, хоча ім'я змінної може бути переприсвоєно посиланню на інший об'єкт, відмінний від того, якому це ім'я було присвоєно на початку. Наприклад:

```
a = 2025
b = a
print(f'id(a):\t{id(a)}', f'id(b):\t{id(b)}', sep = '\n')
```

```
id(a):	139674161292496
id(b):	139674161292496
```

```
[2] a = 2.27
print(a, b)
print(f'id(a):\t{id(a)}', f'id(b):\t{id(b)}', sep = '\n')
```

```
2.27 2025
id(a):	139674183329520
id(b):	139674161292496
```

Змінювані (Mutable) та Незмінювані (Immutable) типи даних

Більш зручний спосіб перевірки на те, чи дві змінні посилаються на той самий об'єкт - це використання оператора is.

```
[3] c = 2.27
    d = b

    print(a is b, d is b, a is c)
```

```
False True False
```

```
[4] print(c is a)
```

```
False
```

Змінювані (Mutable) та Незмінювані (Immutable) типи даних

```
[14] a = [2024, 2025]
     b = a
     print(f'id(a):\t{id(a)}', f'id(b):\t{id(b)}', sep = '\n')
```

```
id(a):	139673270630016
id(b):	139673270630016
```

```
[15] b[0]=-1
     a
```

```
[-1, 2025]
```

Змінювані (Mutable) та Незмінювані (Immutable) типи даних

```
[35] import numpy as np

     rng = np.random.default_rng(seed = 42)

     arr = rng.integers(-5,12, 10)
     arr
```

```
array([-4,  8,  6,  2,  2,  9, -4,  6, -2, -4])
```

```
[36] arr1 = sorted(arr)

     print(arr1, arr, arr1 is arr, sep='\n')
```

```
[-4, -4, -4, -2, 2, 2, 6, 6, 8, 9]
[-4  8  6  2  2  9 -4  6 -2 -4]
False
```

```
[37] arr2 = arr.sort()
     arr
```

```
array([-4, -4, -4, -2,  2,  2,  6,  6,  8,  9])
```

```python
def f(a, b, l=[]):
    l.append(a)
    l.append(b)
    return l

f(1,2)
f(6, -1)
print(f(0,5))  # ?
```

```python
def f(a, b, l=[]):
    l.append(a)
    l.append(b)
    return l

f(1,2)
f(6, -1)
print(f(0,5))  # [1, 2, 6, -1, 0, 5]
```

```python
def f(a, b, d={}):
    d[a]=b
    return d

f(1,2)
f(6, -1)
print(f(0,5))  # ?
```

```python
def f(a, b, d={}):
    d[a]=b
    return d

f(1,2)
f(6, -1)
print(f(0,5)) # {1:2, 6:-1, 0:5}
```

Синтаксичний Цукор

Синтаксичний Цукор

```
[66] x = y = z = -1
```

```
[68] x += 1
     x, y = y, x
     print(x,y)
```

```
[70] print('ok') if x else print('error')  # ?
```

```
[ ] a, b, c = x * (-1), 'hello', -4.5
```

```
[69] a, *other = x + 1, 'hello', -4.5
     other
```

```
['hello', -4.5]
```

```
[54] *first, a = 1,2,3,4,5,6,7,8
```

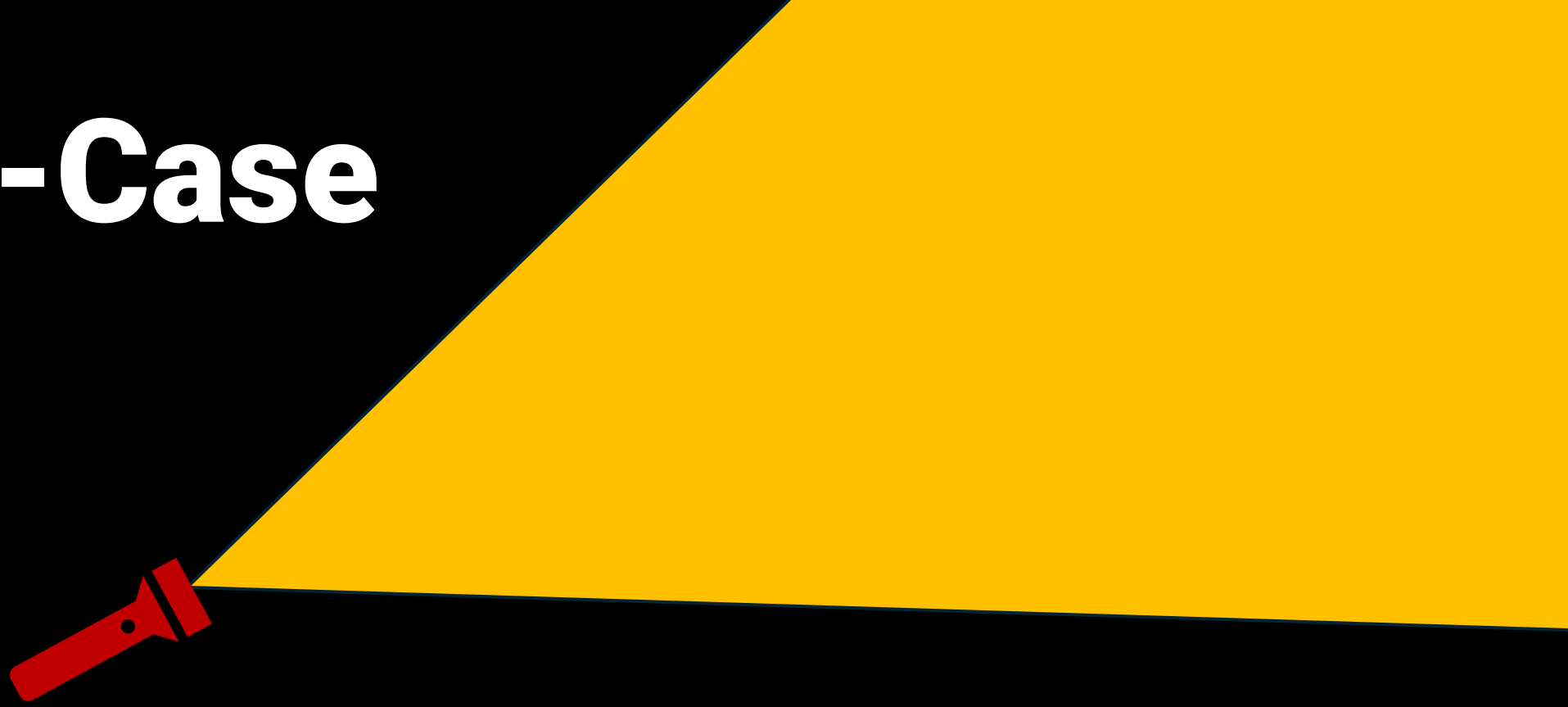
```python
[71] dict1 = {"a": 1, "b": 2}
     dict2 = {"b": 3, "c": 4}

     result = {**dict1, **dict2}  # розпакування
     print(result)  # {'a': 1, 'b': 3, 'c': 4}
```

```python
[72] dict1 = {"a": 1, "b": 2}
     dict2 = {"b": 3, "c": 4}

     result = dict1 | dict2  # Об'єднання словників
     print(result)  # {'a': 1, 'b': 3, 'c': 4}
```

```python
[73] dict1 = {"a": 1, "b": 2}
     dict2 = {"b": 3, "c": 4}

     dict1 |= dict2  # Оновлюємо dict1
     print(dict1)  # {'a': 1, 'b': 3, 'c': 4}
```

Синтаксичний Цукор

Match-Case

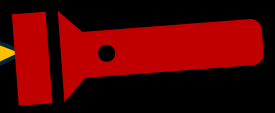


Match-Case

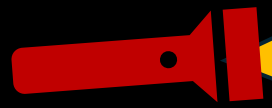
```
command = "start"

match command:
case "start":
    print("Запуск програми")
case "stop":
    print("Зупинка програми")
case "pause":
    print("Пауза")
case _:
    print("Невідома команда")
```

Match-Case

```
status_code = 404

match status_code:
case 200 | 201:
    print("Успішно")
case 400 | 404:
    print("Помилка клієнта")
case 500:
    print("Помилка сервера")
case _:
    print("Невідомий код")
```



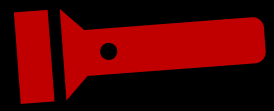
Match-Case



```
user = ("John", "admin")
# Розпакування
match user:
case (name, "admin"):
    print(f"Адмін {name}")
case (name, "user"):
    print(f"Користувач {name}")
case _:
    print("Невідомий користувач")
```

Match-Case

```
data = {"type": "error", "code": 500}

match data:
case {"type": "error", "code": code} if code >= 500:
    print(f"Критична помилка: {code}")
case {"type": "error", "code": code}:
    print(f"Помилка: {code}")
case _:
    print("Невідомий формат")
```



Q&A